



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

درس مهندسی اینترنت

فاز یک پروژه «توزیع مبتنی بر Multicast در GNS3»

عنوان گزارش:

پیاده‌سازی عملی IGMP Proxy و مسیریابی Multicast در توپولوژی دو-بخشی
Upstream/Downstream

دانشجویان:

سپهر رجبی

۴۰۱۳۶۲۳۰۰۸

سنا باقری

۴۰۱۲۳۶۳۴۲۵

پاییز ۱۴۰۴

۱-۱ مقدمه و هدف فاز اول

در شبکه‌های مبتنی بر IP معمولاً سه روش اصلی برای رساندن ترافیک وجود دارد: Unicast (ارسال یک‌به‌یک)، Broadcast (ارسال برای همه‌ی میزبان‌های یک سگمنت) و Multicast (ارسال برای مجموعه‌ای از گیرنده‌ها که به صورت گروه تعریف شده‌اند). در کاربردهایی مثل IPTV، کلاس آنلاین و پخش زنده، اگر برای هر گیرنده یک جریان Unicast جداگانه ارسال شود، پهنای باند شبکه و توان پردازشی روترها به سرعت هدر می‌رود؛ از طرف دیگر Broadcast هم تمام میزبان‌ها را درگیر می‌کند، حتی آن‌هایی که علاقه‌ای به ترافیک ندارند. Multicast تعادلی بین این دو است: فرستنده تنها یک کپی از بسته را به یک آدرس گروهی می‌فرستد و روترها فقط در مسیرهایی که واقعاً گیرنده‌ی عضو آن گروه دارد، بسته را تکثیر می‌کنند.

برای اینکه روتر بداند کدام میزبان‌ها عضو کدام گروه Multicast هستند، از پروتکل IGMP در سمت میزبان‌ها و از مکانیزم‌هایی مثل IGMP Proxy/IGMP Snooping در سمت روترها و سوییچ‌ها استفاده می‌شود. در سناریوی این پروژه، شبکه به دو بخش Upstream (سمت فرستنده) و Downstream (سمت گیرنده‌ها) تقسیم شده است و روتر مرکزی باید بتواند عضویت‌های Multicast را از سمت Downstream جمع‌آوری و بر اساس آن، ترافیک را از سمت Upstream به درستی فورواردهد.

• در فاز اول، تمرکز بر طراحی توپولوژی، راه‌اندازی IGMP Proxy، عضویت در گروه‌های Multicast، مشاهده‌ی بسته‌های UDP Multicast/IGMP در Wireshark و پیاده‌سازی یک «Multicast Ping سطح برنامه» است.

• در فاز دوم، همین زیرساخت شبکه و کدها برای پیاده‌سازی یک برنامه‌ی گرافیکی Canvas اشتراکی استفاده می‌شود که نقاط ترسیم‌شده روی یک بوم، به صورت چندپخشی بین چند کاربر توزیع می‌شود.

بنابراین تمرکز این گزارش بر فاز اول است و نشان می‌دهد چگونه از یک توپولوژی خام در GNS3، قدم به قدم به سیستم عملی زیر می‌رسیم:

• امکان عضویت میزبان‌ها در گروه‌های Multicast و مشاهده‌ی پیام‌های IGMP (گزارش عضویت و پرس‌وجو).

• فعال‌بودن IGMP در شبکه و نقش روتر به عنوان IGMP Querier.

• پیاده‌سازی مسیریابی چندپخشی بین یک شبکه‌ی Upstream و یک شبکه‌ی Downstream با استفاده از IGMP Proxy.

• پیاده‌سازی یک Application-level Multicast Ping با دو اسکریپت Python Sender/Receiver، به‌طوری‌که هم مسیر رفت (Multicast) و هم مسیر برگشت (Unicast) تست شود.

• قابلیت ردیابی و تحلیل مسیر توزیع ترافیک چندپخش با ابزارهایی مانند Wireshark و جدول Multicast Routing روی روتر.

در ادامه، ساختار شبکه، تنظیمات روتر، منطق برنامه‌ها و خروجی‌های Wireshark به‌گونه‌ای ارائه شده‌اند که رشته‌ی منطقی «از تعریف مفاهیم تا مشاهده‌ی بسته‌ی واقعی روی سیم» کاملاً قابل دنبال‌کردن باشد و زیرساخت به‌صورت مستقیم در فاز دوم نیز قابل استفاده بماند.

۲-۱ خلاصه نیازمندی‌های فاز اول از دید داکيومنت پروژه

چک‌لیست رسمی فاز اول در صورت‌مسئله چند محور اصلی را مشخص کرده است. در این گزارش، هر بخش بعدی به‌طور مشخص نشان می‌دهد که هر بند چگونه در توپولوژی و پیاده‌سازی عملی محقق شده است.

۱-۲-۱ توپولوژی و ساختار شبکه

بر اساس صورت‌مسئله، ساختار پایه‌ی شبکه باید ویژگی‌های زیر را داشته باشد:

- وجود یک روتر مرکزی که IGMP Proxy روی آن فعال است.
- یک شبکه‌ی Upstream شامل دست‌کم یک فرستنده (Sender).
- یک شبکه‌ی Downstream شامل حداقل دو میزبان گیرنده (Receivers).
- جداسازی منطقی و لایه‌دو (LAN/Switch‌های جداگانه) بین Upstream و Downstream برای مشاهده‌ی واضح‌تر رفتار IGMP روی سگمنت گیرنده‌ها.

۲-۲-۱ نیازمندی‌های IGMP و Proxy IGMP

از منظر پروتکلی، فاز اول باید موارد زیر را پوشش دهد:

- میزبان‌های گیرنده باید بتوانند در Multicast Group ها عضو شوند و IGMP Membership Report ارسال کنند.
- روتر باید نقش IGMP Proxy را بر عهده بگیرد؛ یعنی:

– یک Upstream Interface و یک یا چند Downstream Interface تعریف شود.

- عضویت‌های Multicast از Downstream جمع‌آوری و به Upstream پروکسی شوند.
- ترافیک Multicast پس از دریافت از Upstream، روی Downstream بازپخش شود.
- امکان تست و مشاهده:

- IGMP Membership Report و IGMP Membership Query.
 - جدول مسیریابی چندپخش (Multicast Routing Table یا MFC) روی روتر.
 - بسته‌های Multicast با tcpdump/Wireshark.
 - نوعی Multicast Ping (در صورت محدودیت سیستم عامل، به صورت پیاده‌سازی سطح برنامه).
- در بخش‌های بعدی نشان داده می‌شود که هر یک از این بندها، روی توپولوژی پیشنهادی و با کمک اسکریپت‌های Python دقیقاً چگونه پیاده‌سازی و راستی‌آزمایی شده‌اند.
- ۱-۳ طراحی توپولوژی و انتخاب ابزارها**

۱-۳-۱ پلتفرم، نسخه‌ها و محدودیت‌های سخت‌افزاری

- برای پیاده‌سازی عملی این فاز، ترکیب زیر استفاده شده است:
- GNS3 به عنوان بستر اصلی شبیه‌سازی توپولوژی.
 - MikroTik RouterOS v7 (به صورت QEMU Guest در GNS3) به عنوان روتر مرکزی.
 - GNS3 Built-in Ethernet Switch به عنوان Switch لایه ۲، بدون نیاز به تصویر IOS جداگانه.
 - سه ماشین مجازی Windows 7 Lite در VMware Workstation:

- **PC۱:** فرستنده (Sender) در شبکه‌ی Upstream.
 - **PC۲:** گیرنده‌ی ۱ (Receiver 1) در شبکه‌ی Downstream.
 - **PC۳:** گیرنده‌ی ۲ (Receiver 2) در شبکه‌ی Downstream.
- با توجه به محدودیت RAM سیستم میزبان (مصرف حدود ۱ تا ۲ گیگابایت برای هر VM ویندوزی)، تعداد میزبان‌ها به سه عدد محدود شده است تا امکان اجرای هم‌زمان GNS3، GNS3 و VMware فراهم باشد.

۱-۳-۲ توپولوژی منطقی و فیزیکی

ساختار نهایی توپولوژی (شکل ۱) به صورت زیر است:

• سمت Upstream:

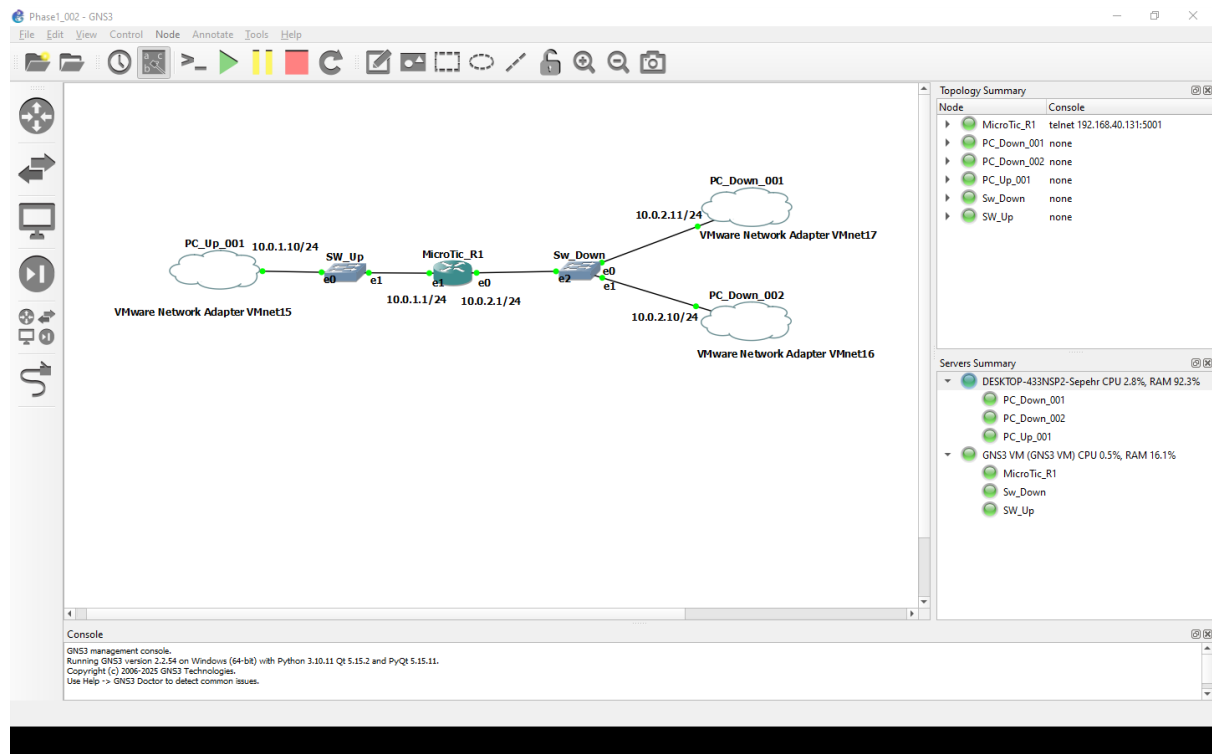
- PC1 (Sender) روی VMnet15 در حالت Host-only.
- یک Cloud در GNS3 متصل به VMnet15 VMware Network Adapter.
- یک Ethernet Switch داخلی GNS3 بین Cloud و اینترفیس ether2 روتر.

• سمت Downstream:

- PC2 روی VMnet16 و PC3 روی VMnet17 (هر دو در حالت Host-only).
- دو Cloud جدا در GNS3:
- * یکی متصل به VMnet16 (برای PC2).
- * دیگری متصل به VMnet17 (برای PC3).
- هر دو Cloud به یک Ethernet Switch دوم در GNS3 متصل شده‌اند و این Switch به عنوان LAN مشترک Downstream به اینترفیس ether1 روتر متصل است.

• روتر مرکزی:

- MikroTik RouterOS v7 با دو اینترفیس:
- * ether2: متصل به Upstream Switch (سمت فرستنده).
- * ether1: متصل به Downstream Switch (سمت گیرنده‌ها).
- IGMP Proxy روی روتر فعال شده و ether2 به عنوان Upstream Interface و ether1 به عنوان Downstream Interface پیکربندی شده است؛ دقیقاً مطابق بخش «پیکربندی Proxy IGMP» از صورت‌مسئله.



شکل ۱: توپولوژی نهایی فاز یک در GNS3: یک روتر MikroTik، دو Switch داخلی، و سه PC ویندوزی متصل از طریق Cloud و VMnet.

۴-۱ پیکربندی آدرس‌دهی و تست اتصال Unicast

مطابق صورت‌مسئله، قبل از ورود به بحث IGMP/Multicast باید زیرساخت IPv4 Unicast بین تمام نودها به‌درستی کار کند. در این بخش نشان داده می‌شود که این پیش‌نیاز چگونه پیاده‌سازی و تست شده است.

۱-۴-۱ پیکربندی آدرس روی روتر MikroTik

روی روتر MikroTik، دو آدرس IPv4 طبق طرح زیر تنظیم شد:

• ether2 (سمت Upstream):

– آدرس: 10.0.1.1/24

– متصل به PC1 (Sender) از طریق Switch-Up.

• ether1 (سمت Downstream):

– آدرس اولیه: 10.0.2.1/24

– در بخشی از تست‌ها، برای سهولت Capture روی کارت شبکه‌ی VMware، زیرشبکه‌ی سمت Downstream به 192.168.237.0/24 و آدرس روتر به 192.168.237.132 تغییر داده شد؛ به همین دلیل در خروجی Wireshark، هم 10.0.2.1 و هم 192.168.237.132 به‌عنوان مبدأ بسته‌های IGMP Query دیده می‌شوند.

فرمان‌های اصلی در MikroTik CLI:

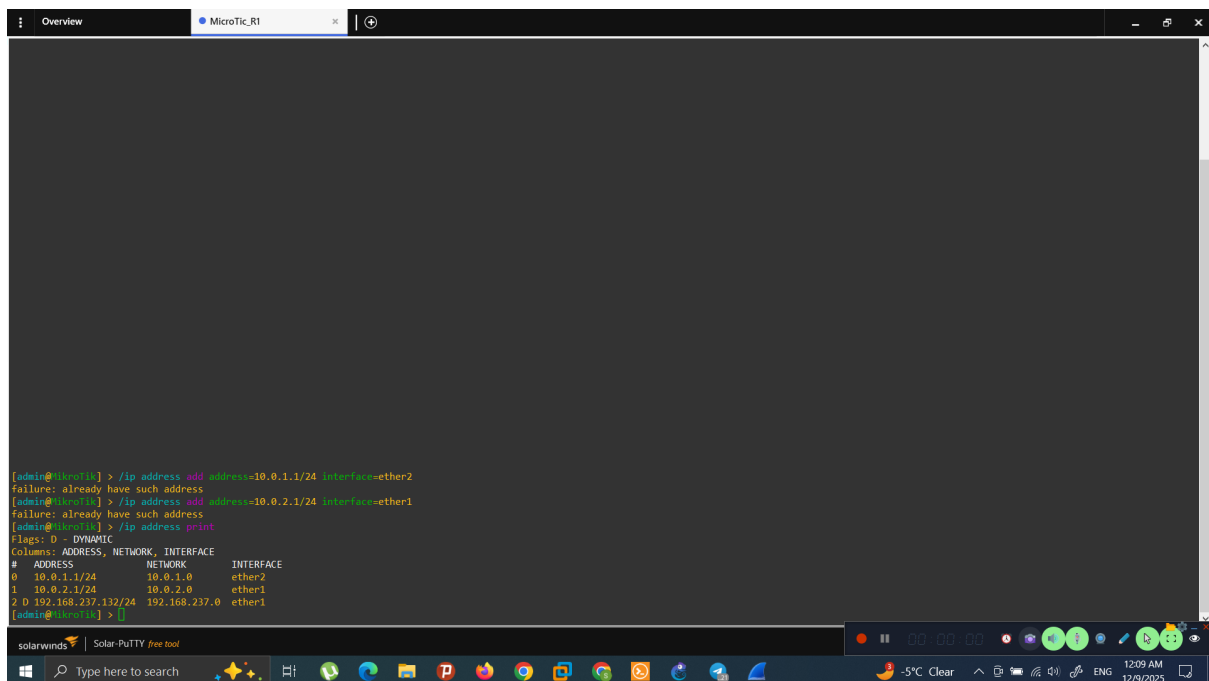
```
/interface print
```

```
/ip address add address=10.0.1.1/24 interface=ether2
```

```
/ip address add address=10.0.2.1/24 interface=ether1
```

```
/ip address print
```

شکل ۲ نمونه‌ی تنظیم آدرس‌ها روی روتر را نشان می‌دهد.



شکل ۲: پیکربندی آدرس‌دهی IPv4 روی روتر MikroTik برای اینترفیس‌های ether1 و ether2.

۲-۴-۱ پیکربندی آدرس روی میزبان‌ها

هر سه PC ویندوزی با Static IP پیکربندی شده‌اند:

• **Sender PC۱** روی VMnet15:

– IP :10.0.1.10

– Mask :255.255.255.0

– Default Gateway :10.0.1.1

• **Receiver PC۱** روی VMnet16:

– IP :10.0.2.10

– Mask :255.255.255.0

– Default Gateway :10.0.2.1

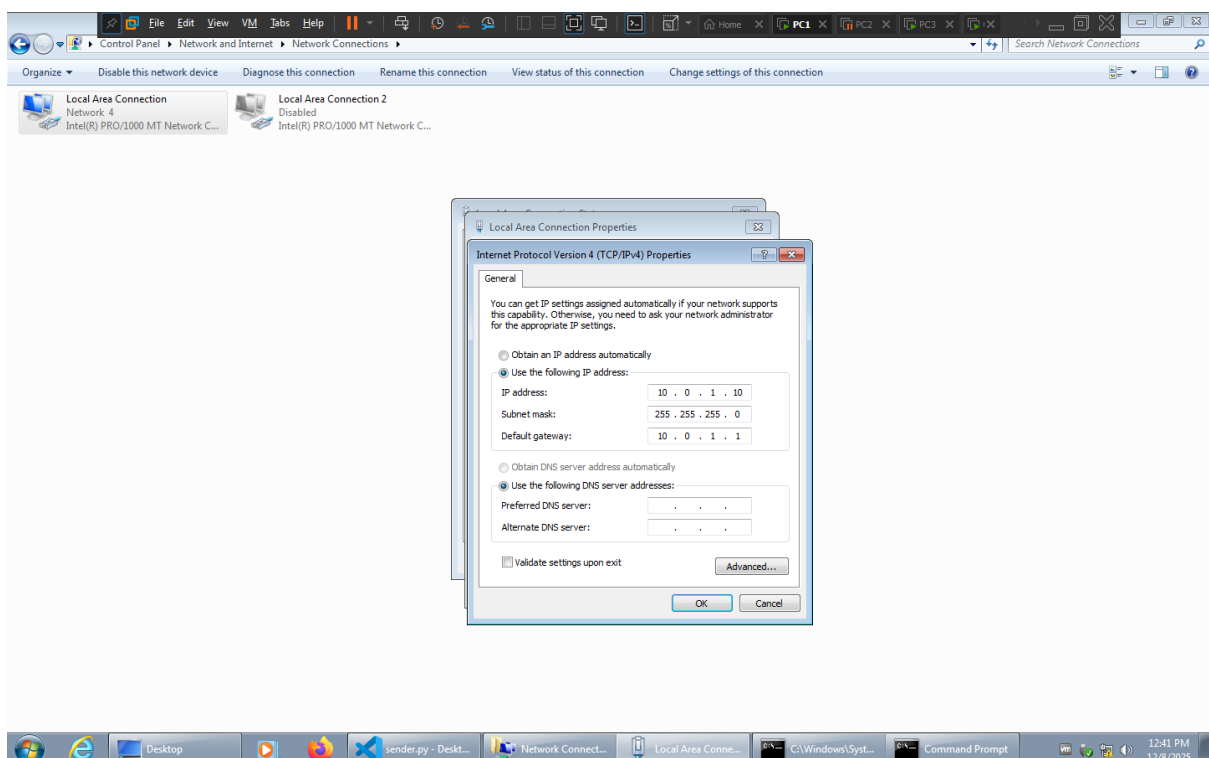
• **Receiver PC۳** روی VMnet17:

– IP :10.0.2.11

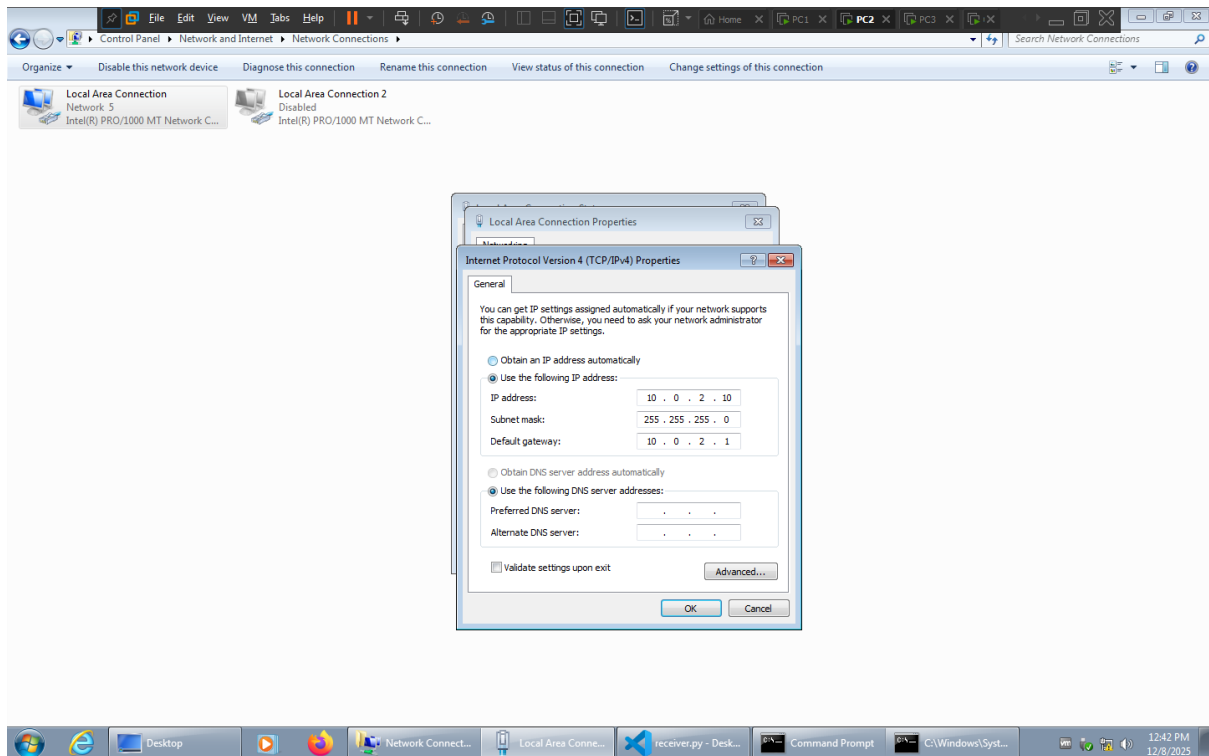
– Mask :255.255.255.0

– Default Gateway :10.0.2.1

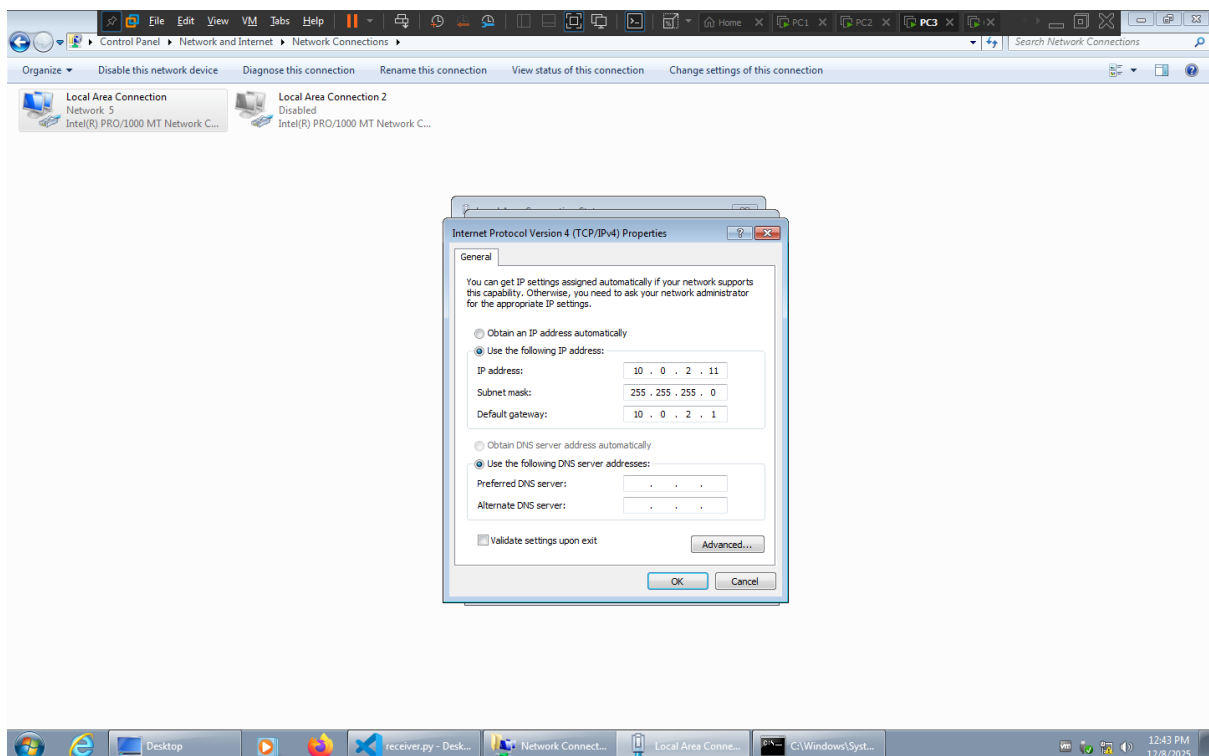
نمونه‌ی تنظیمات روی هر PC در شکل‌های ۳ تا ۵ آمده است.



شکل ۳: پیکربندی آدرس‌دهی IPv4 روی Sender. PC1



شکل ۴: پیکربندی آدرس‌دهی IPv4 روی Receiver PC2.



شکل ۵: پیکربندی آدرس‌دهی IPv4 روی Receiver PC3.

۳-۴-۱ تست اتصال Unicast بین همه‌ی نودها

در راستای بند «تست اتصال اولیه» از صورت‌مسئله، پس از تنظیم IP و Gateway، Ping‌های زیر انجام شد:

• از PC1 به آدرس روتر سمت Upstream (10.0.1.1) و همچنین به گیرنده‌ها (10.0.2.10 و 10.0.2.11).

• از PC2 و PC3 به آدرس روتر سمت Downstream (10.0.2.1) و نیز به یکدیگر.

• از روتر به هر سه PC در هر دو شبکه.

خروجی این Ping‌ها (شکل‌های ۶ تا ۸) نشان می‌دهد که مسیریابی Unicast بین تمام نودها به درستی کار می‌کند و زیرساخت برای فعال‌سازی Multicast آماده است.

```

Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\PC1.OPREKIN-PC>ping 10.0.1.1

Pinging 10.0.1.1 with 32 bytes of data:
Reply from 10.0.1.1: bytes=32 time=2ms TTL=64
Reply from 10.0.1.1: bytes=32 time=1ms TTL=64
Reply from 10.0.1.1: bytes=32 time=2ms TTL=64
Reply from 10.0.1.1: bytes=32 time=1ms TTL=64

Ping statistics for 10.0.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 2ms, Average = 1ms

C:\Users\PC1.OPREKIN-PC>ping 10.0.2.1

Pinging 10.0.2.1 with 32 bytes of data:
Reply from 10.0.2.1: bytes=32 time<1ms TTL=64
Reply from 10.0.2.1: bytes=32 time=1ms TTL=64
Reply from 10.0.2.1: bytes=32 time=1ms TTL=64
Reply from 10.0.2.1: bytes=32 time=1ms TTL=64

Ping statistics for 10.0.2.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\PC1.OPREKIN-PC>ping 10.0.2.10

Pinging 10.0.2.10 with 32 bytes of data:
Reply from 10.0.2.10: bytes=32 time=3ms TTL=127
Reply from 10.0.2.10: bytes=32 time=1ms TTL=127
Reply from 10.0.2.10: bytes=32 time=3ms TTL=127
Reply from 10.0.2.10: bytes=32 time=3ms TTL=127

Ping statistics for 10.0.2.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 3ms, Average = 2ms

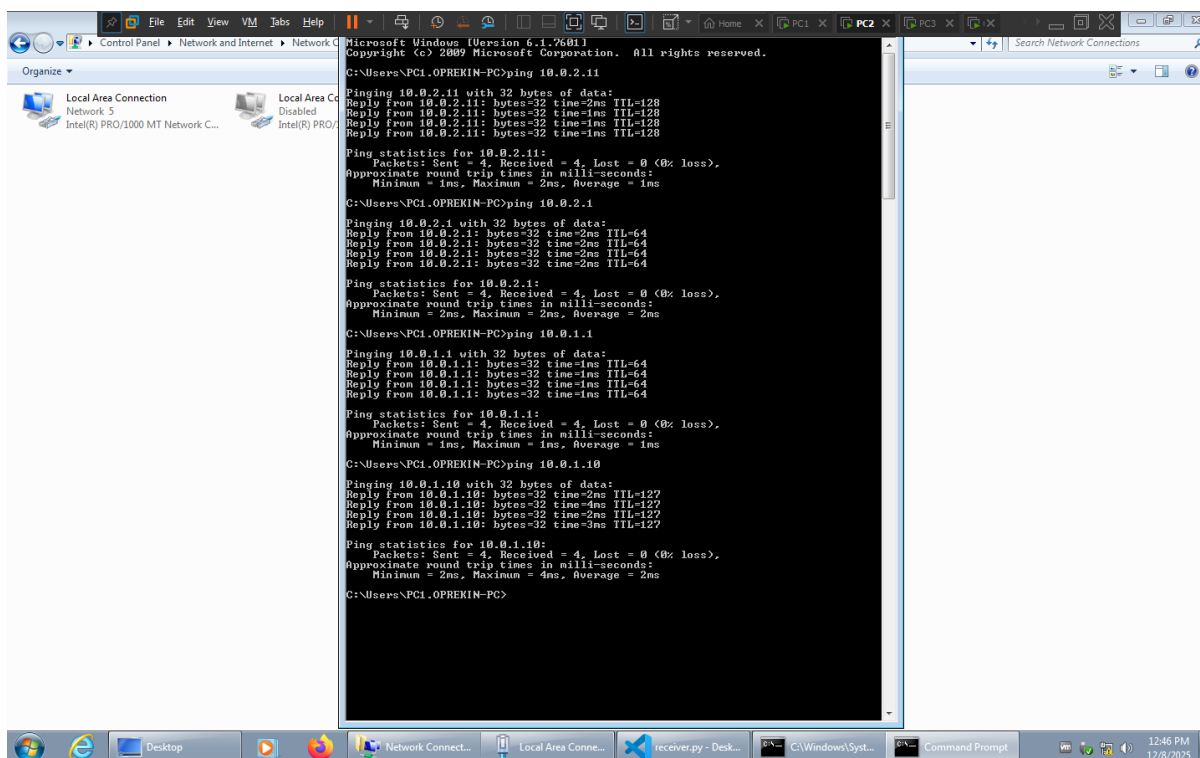
C:\Users\PC1.OPREKIN-PC>ping 10.0.2.11

Pinging 10.0.2.11 with 32 bytes of data:
Reply from 10.0.2.11: bytes=32 time=2ms TTL=127
Reply from 10.0.2.11: bytes=32 time=2ms TTL=127
Reply from 10.0.2.11: bytes=32 time=3ms TTL=127
Reply from 10.0.2.11: bytes=32 time=3ms TTL=127

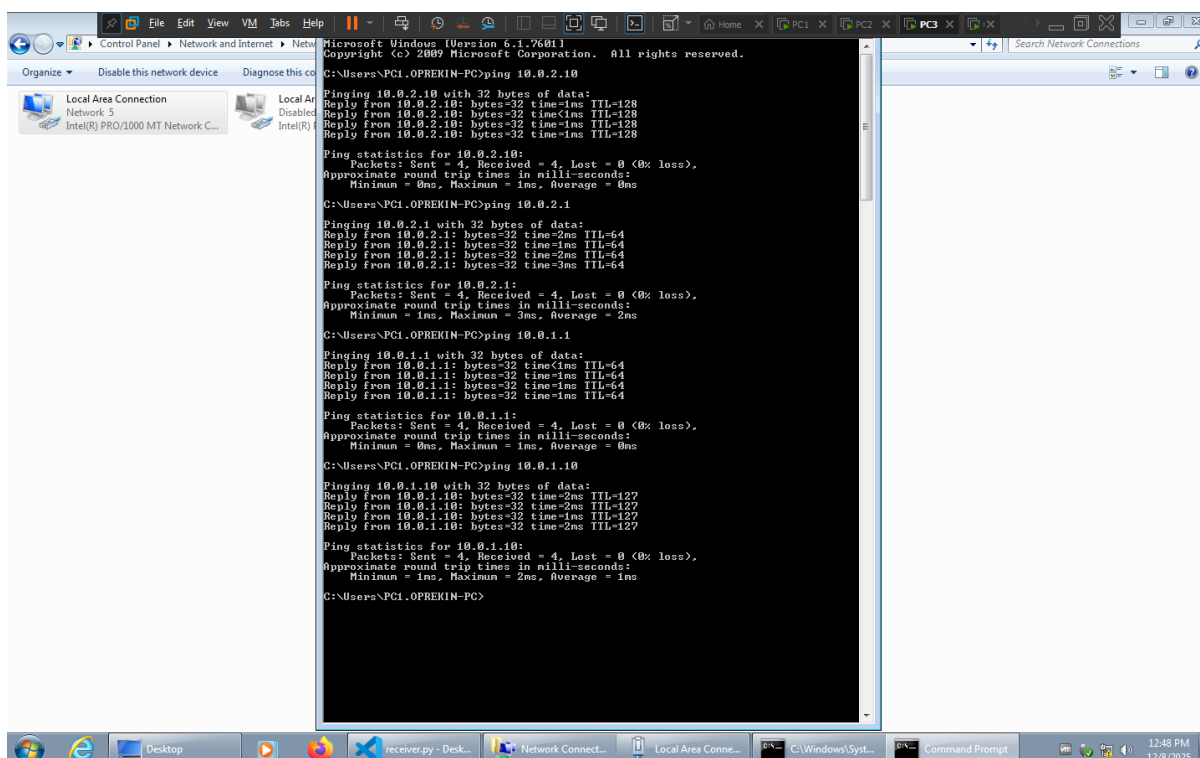
Ping statistics for 10.0.2.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 3ms, Average = 2ms

C:\Users\PC1.OPREKIN-PC>_
    
```

شکل ۶: تست Ping از PC1 به روتر و گیرنده‌ها.



شکل ۷: تست Ping از PC2.



شکل ۸: تست Ping از PC3.

۵-۱ فعال‌سازی و پیکربندی Proxy IGMP روی MikroTik

بر اساس بند «پیکربندی Proxy IGMP روی روتر» در صورت مسئله، لازم است روتر به گونه‌ای تنظیم شود که:

- عضویت‌های IGMP را از سمت Downstream جمع‌آوری کند.
- این عضویت‌ها را به سمت Upstream پروکسی کند.
- ترافیک Multicast را از Upstream دریافت و روی Downstream بازپخش نماید.

در این سناریو:

• ether2 به عنوان Upstream Interface (سمت فرستنده / شبکه‌ی 10.0.1.0/24) در نظر گرفته شده است.

• ether1 به عنوان Downstream Interface (سمت گیرنده‌ها / شبکه‌ی 10.0.2.0/24 و در برخی تست‌ها 192.168.237.0/24) در نظر گرفته شده است.

فرمان‌های اصلی برای فعال‌سازی IGMP Proxy:

```
/routing igmp-proxy interface add interface=ether2 upstream=yes
/routing igmp-proxy interface add interface=ether1 upstream=no

/routing igmp-proxy interface print status
```

نمونه‌ای از وضعیت اینترفیس‌ها پس از فعال‌سازی:

```
[admin@MikroTik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0  U interface=ether2 threshold=1 alternative-subnets="" upstream=yes
source-ip-address=10.0.1.1 rx-bytes=1155 rx-packets=21
tx-bytes=0 tx-packets=0

1  interface=ether1 threshold=1 alternative-subnets="" upstream=no
querier=yes source-ip-address=192.168.237.132
rx-bytes=0 rx-packets=0 tx-bytes=1155 tx-packets=21
```

در چند اجرای متوالی این دستور، مقدار rx-packets روی ether2 از ۰ به ۲۱ و سپس بیشتر، و مقدار tx-packets روی ether1 به همان نسبت افزایش یافته است. این رفتار نشان می‌دهد که هر بسته‌ی چندپخشی که از سمت Upstream وارد روتر می‌شود، یک کپی روی Downstream به سمت گیرنده‌ها ارسال می‌گردد.

از این خروجی مشخص است که:

- اینترفیس ether2 با علامت U به‌عنوان Upstream مشخص شده و در حال دریافت بسته‌ها (rx-packets) از سمت شبکه‌ی Upstream است.
- اینترفیس ether1 نقش IGMP Querier روی شبکه‌ی Downstream را دارد (querier=yes) و پرس‌وجوهای IGMP را روی این سگمنت ارسال می‌کند.

```

[admin@Mikrotik] > /routing igmp-proxy print
quick-leave: no
query-interval: 2m5s
query-response-interval: 10s
[admin@Mikrotik] > /routing igmp-proxy interface add interface=ether2 upstream=yes
failure: already have such interface
[admin@Mikrotik] > /routing igmp-proxy interface add interface=ether1 upstream=no
failure: already have such interface
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=0 rx-packets=0 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=0 tx-packets=0
[admin@Mikrotik] >
    
```

شکل ۹: فعال‌سازی IGMP Proxy روی روتر MikroTik و نمایش وضعیت اینترفیس‌های Downstream/Upstream.

۱-۶ برنامه‌ی Sender/Receiver و شبیه‌سازی Ping Multicast

طبق صورت‌مسئله، در فاز اول باید امکان «Multicast Ping» نیز بررسی شود. در Windows 7، ابزار استاندارد ping از ارسال ICMP Echo به آدرس‌های Multicast پشتیبانی نمی‌کند؛ بنابراین در این پروژه به‌جای ICMP، یک **Application-level Multicast Ping** مبتنی بر UDP طراحی و پیاده‌سازی

شده است تا همان کارکرد آموزشی را داشته باشد: یعنی «ارسال یک پیام به صورت Multicast و دریافت پاسخ از همه‌ی گیرنده‌های عضو».

ایده‌ی کلی این بخش که مستقیماً نیازمندی‌های «برنامه‌ی ارسال‌کننده/گیرنده» در فاز دوم را نیز پوشش می‌دهد به صورت زیر است:

• Sender روی PC۱:

- در هر چرخه، یک متن (مثلاً «Hello from multicast PC-Up!») را به صورت UDP Multicast روی $239.1.1.1 = \text{Group}$ و $5000 = \text{Port}$ ارسال می‌کند.
- سپس برای مدت کوتاهی روی همان Port منتظر دریافت پیام‌های PING از گیرنده‌ها می‌ماند.
- به ازای هر PING دریافتی، یک پیام PONG به صورت Unicast به همان گیرنده بازمی‌گرداند.

• Receiver روی PC۲ و PC۳:

- به $239.1.1.1 = \text{Multicast Group}$ روی $5000 = \text{Port}$ ملحق می‌شود و این همان بند «عضویت در Group Multicast» در صورت مسئله است.
- هر بسته‌ی دریافتی را چاپ می‌کند:
- * اگر متن PONG باشد، به عنوان پاسخ Ping ثبت می‌شود.
- * در غیر این صورت، متن را پیام Sender فرض کرده و در پاسخ یک پیام PING به صورت Unicast به Sender می‌فرستد.

به این ترتیب، هر چرخه شامل:

- یک ارسال Multicast از Upstream به همه‌ی گیرنده‌ها در Downstream.
- چند پاسخ Unicast برگشتی (PING/PONG) بین گیرنده‌ها و فرستنده.

این رفتار عملاً نقش یک Multicast Ping را برای تست همزمان مسیر رفت (Multicast) و برگشت (Unicast) ایفا می‌کند و در Wireshark نیز به وضوح قابل مشاهده است (فریم‌های UDP Multicast و Unicast که در ادامه تحلیل می‌شوند).

۱-۶-۱ کد کامل Receiver روی PC۲ و PC۳

```
import socket

import struct

MCAST_GRP = '239.1.1.1'
MCAST_PORT = 5000

# Create UDP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)

# Allow reuse of address (helps if you restart quickly)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Bind to all interfaces on the multicast port (for both multicast + unicast on the
sock.bind(('', MCAST_PORT))

# Join the multicast group on all interfaces
mreq = struct.pack(
    '4sl',
    socket.inet_aton(MCAST_GRP),
    socket.INADDR_ANY
)

sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)

print(f"[RECEIVER] Listening on multicast group {MCAST_GRP}:{MCAST_PORT} ...")
print("[RECEIVER] - On multicast TEXT: print it and send PING (unicast) to sender.")
print("[RECEIVER] - On unicast PONG from sender: print that PONG was received.\n")

while True:
    data, addr = sock.recvfrom(4096)
    text = data.decode(errors='ignore').strip()
```

```

src_ip, src_port = addr

print(f"[RECEIVER] From {src_ip}:{src_port} -> {repr(text)}")

# 1) If we receive PONG from the sender (unicast reply)
if text.upper() == "PONG":
    print(f"[RECEIVER] <<< Received PONG from sender {src_ip}:{src_port}\n")
    continue

# 2) Ignore control PING if ever used in another direction (optional)
if text.upper() == "PING":
    # In this scenario we don't respond to multicast PING;
    # we only send PING back when we receive TEXT.
    continue

# 3) Otherwise treat it as application TEXT from sender (multicast)
print(f"[RECEIVER] Handling multicast TEXT: {text}")

# Now "ping" the sender back with a unicast PING
ping_msg = "PING"
sock.sendto(ping_msg.encode(), addr)
print(f"[RECEIVER] >>> Sent unicast PING to sender {src_ip}:{src_port}\n")

```

در این کد:

- ابتدا یک سوکت UDP ساخته می شود که توانایی دریافت هم Multicast و هم Unicast روی پورت 5000 را دارد (bind("", 5000)).
- با IP_ADD_MEMBERSHIP، سیستم عامل در گروه 239.1.1.1 عضو می شود. همین فراخوانی باعث ایجاد IGMP Membership Report برای این گروه و ارسال آن به روتر می شود.

• حلقه‌ی `while True` یک «حلقه‌ی رویداد» ساده است که هر بسته‌ی دریافتی را `decode` کرده و بسته به محتوای آن تصمیم می‌گیرد:

- اگر "PONG" باشد، یعنی پاسخ از Sender است و فقط لاگ می‌شود.
- اگر "PING" باشد، در این سناریو نادیده گرفته می‌شود تا از حلقه‌های غیرضروری جلوگیری شود.
- در غیر این صورت، آن را متن آموزشی از Sender در نظر می‌گیرد، روی کنسول چاپ می‌کند و به صورت Unicast یک "PING" به همان آدرس بازمی‌گرداند.

به این ترتیب، رفتار Receiver تقریباً شبیه یک «Multicast Echo Client» است: هر متن چندپخشی را دریافت و با یک پیام کنترل PING جواب می‌دهد.

۱-۶-۲ کد کامل Sender روی PC

```
import socket
import struct
import time

MCAST_GRP = '239.1.1.1'
MCAST_PORT = 5000

TEXT_MESSAGE = "Hello multicast from PC-Up!"

# Create UDP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)

# Allow address reuse
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Bind so we can receive unicast PINGs on this port
sock.bind(('', MCAST_PORT))
```

```
# Set multicast TTL (must be >= 2 to cross MikroTik)
ttl = struct.pack('b', 16)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, ttl)

print(f"[SENDER] Sending to multicast group {MCAST_GRP}:{MCAST_PORT}")
print("[SENDER] Each cycle:")
print("  1) Send a multicast TEXT to all receivers.")
print("  2) Wait for unicast PINGs from receivers.")
print("  3) For each PING, show it and send back a unicast PONG.\n")

while True:
    # 1) Send multicast TEXT
    sock.sendto(TEXT_MESSAGE.encode(), (MCAST_GRP, MCAST_PORT))
    print(f"[SENDER] >>> Sent TEXT to {MCAST_GRP}:{MCAST_PORT} -> {TEXT_MESSAGE!r}")

    # 2) Wait for PINGs from receivers (short time window)
    sock.settimeout(0.5) # 0.5s per recv
    ping_count = 0
    pong_count = 0
    start = time.time()

    while True:
        try:
            data, addr = sock.recvfrom(4096)
        except socket.timeout:
            # No more data in this cycle
            break

    text = data.decode(errors='ignore').strip()
    src_ip, src_port = addr
```

```

if text.upper() == "PING":
    ping_count += 1
    print(f"[SENDER] <<< Received PING from {src_ip}:{src_port}")

# 3) Reply with PONG (unicast) to that receiver
pong_msg = "PONG"
sock.sendto(pong_msg.encode(), addr)
pong_count += 1
print(f"[SENDER] >>> Sent PONG to {src_ip}:{src_port}")
else:
    print(f"[SENDER] (other) from {src_ip}:{src_port}: {repr(text)}")

# Hard limit so we don't loop too long per cycle
if time.time() - start > 1.0:
    break

print(f"[SENDER] Cycle complete: PINGs received = {ping_count}, "
      f"PONGs sent = {pong_count}\n")

# Small delay before next cycle
time.sleep(2)

```

در این کد:

- Sender هم‌زمان دو نقش دارد: هم **فرستنده‌ی Multicast** است (برای متن آموزشی) و هم **سرور Unicast** برای پاسخ به PING‌ها.
- چون سوکت روی همان پورت 5000 bind شده است، بسته‌های PING که از گیرنده‌ها می‌آیند نیز روی همین سوکت دریافت می‌شوند.
- مقدار IP_MULTICAST_TTL برابر ۱۶ انتخاب شده تا بسته‌ی چندپخش‌ی بتواند از روتر عبور

کند (حداقل باید ۲ باشد)، اما عملاً در این توپولوژی کوچک بعد از عبور از روتر، TTL در سمت Downstream برابر ۱۵ دیده می‌شود.

• حلقه‌ی بیرونی `while True` یک «چرخه‌ی پینگ» را پیاده‌سازی می‌کند:

۱. ارسال متن چندپختی به گروه.

۲. باز کردن یک پنجره‌ی زمانی کوتاه (حداکثر حدود ۱ ثانیه) برای دریافت PING ها.

۳. در هر دریافت PING، افزایش شمارنده، چاپ روی کنسول و ارسال یک PONG به همان آدرس.

• گزارش انتهایی هر چرخه (`Cycle complete: PINGs received = ...`) نشان می‌دهد چند گیرنده فعال بوده‌اند و چند پاسخ PONG ارسال شده است.

```

# 2) Wait for PINGs from receivers (short time window)
sock.settimeout(0.5) # 0.5s per recv
ping_count = 0
pong_count = 0
start = time.time()

while True:
    try:
        data, addr = sock.recvfrom(4096)
    except socket.timeout:
        # No more data in this cycle
        break

    text = data.decode(errors='ignore').strip()
    src_ip, src_port = addr

    if text.upper() == "PING":
        ping_count += 1
        print(f"[SENDER] <<< Received PING from {src_ip}:{src_port}")

        # 3) Reply with PONG (unicast) to that
        pong_msg = "PONG"
        sock.sendto(pong_msg.encode(), addr)
        pong_count += 1
        print(f"[SENDER] >>> Sent PONG to {src_ip}:{src_port}")
    else:
        print(f"[SENDER] (other) from {src_ip}:{src_port}: {repr(text)}")

    # Hard limit so we don't loop too long per cycle
    if time.time() - start > 1.0:
        break

    print(f"[SENDER] Cycle complete: PINGs received = {ping_count}, PONGs sent = {pong_count}\n")

    # Small delay before next cycle
    time.sleep(2)
    
```

```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\PC1-OPREKIN-PC\Desktop>python sender.py
[SENDER] Sending to multicast group 239.1.1.1:5000
[SENDER] Each cycle:
1) Send a multicast TEXT to all receivers.
2) Wait for unicast PINGs from receivers.
3) For each PING, show it and send back a unicast PONG.

[SENDER] >>> Sent TEXT to 239.1.1.1:5000 -> 'Hello multicast from PC-Up!'
[SENDER] <<< Received PING from 10.0.2.10:5000
[SENDER] >>> Sent PONG to 10.0.2.10:5000
[SENDER] <<< Received PING from 10.0.2.11:5000
[SENDER] >>> Sent PONG to 10.0.2.11:5000
[SENDER] Cycle complete: PINGs received = 2, PONGs sent = 2

[SENDER] >>> Sent TEXT to 239.1.1.1:5000 -> 'Hello multicast from PC-Up!'
[SENDER] <<< Received PING from 10.0.2.11:5000
[SENDER] >>> Sent PONG to 10.0.2.11:5000
[SENDER] <<< Received PING from 10.0.2.10:5000
[SENDER] >>> Sent PONG to 10.0.2.10:5000
[SENDER] Cycle complete: PINGs received = 2, PONGs sent = 2

[SENDER] >>> Sent TEXT to 239.1.1.1:5000 -> 'Hello multicast from PC-Up!'
[SENDER] <<< Received PING from 10.0.2.11:5000
[SENDER] >>> Sent PONG to 10.0.2.11:5000
    
```

شکل ۱۰: خروجی کامل کنسول Sender هنگام ارسال بسته‌های چندپختی و تبادل پیام‌های PONG/PING.

The screenshot displays a Windows desktop environment. In the background, a code editor (likely Visual Studio Code) shows the source code for a Python script named `receiver.py`. The script is designed to listen on a multicast group (239.1.1.1:5000) and handle incoming messages. It can receive unicast PING messages from 10.0.1.10:5000 and respond with unicast PONG messages. It also handles multicast TEXT messages and can respond with multicast PING messages. The script uses the `socket` module and `struct` for network communication.

In the foreground, a command prompt window titled `C:\Windows\System32\cmd.exe - python receiver.py` shows the output of the script. The output indicates that the receiver is listening on the multicast group and successfully receives and processes several messages, including unicast PING and PONG, and multicast TEXT.

```

14 sock.bind('', MCAST_PORT)
15
16 # Join the multicast group on all interfaces
17 mreq = struct.pack(
18     '4sI',
19     socket.inet_aton(MCAST_GRP),
20     socket.INADDR_ANY
21 )
22 sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
23
24 print(f"[RECEIVER] Listening on multicast group 239.1.1.1:5000 ...")
25 print("[RECEIVER] - On multicast TEXT: print it and send PING unicast to sender.")
26 print("[RECEIVER] - On unicast PONG from sender: print that PONG was received.")
27
28 while True:
29     data, addr = sock.recvfrom(4096)
30     text = data.decode(errors='ignore').strip()
31     src_ip, src_port = addr
32
33     print(f"[RECEIVER] From {src_ip}:{src_port}")
34
35     # 1) If we receive PONG from the sender (unicast)
36     if text.upper() == "PONG":
37         print(f"[RECEIVER] <<< Received PONG from {src_ip}:{src_port}")
38         continue
39
40     # 2) Ignore control PING if ever used in another direction (optional)
41     if text.upper() == "PING":
42         # In this scenario we don't respond to multicast PING;
43         # we only send PING back when we receive TEXT.
44         continue
45
46     # 3) Otherwise treat it as application TEXT from sender (multicast)
47     print(f"[RECEIVER] Handling multicast TEXT: {text}")
48
49     # Now "ping" the sender back with a unicast PING
50     ping_msg = "PING"
51     sock.sendto(ping_msg.encode(), addr)
52     print(f"[RECEIVER] >>> Sent unicast PING to sender {src_ip}:{src_port}\n")
53

```

```

Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\PC1\OPREKIN-PC\Desktop>python receiver.py
[RECEIVER] Listening on multicast group 239.1.1.1:5000 ...
[RECEIVER] - On multicast TEXT: print it and send PING unicast to sender.
[RECEIVER] - On unicast PONG from sender: print that PONG was received.
[RECEIVER] From 10.0.1.10:5000 -> 'Hello multicast from PC-Up!'
[RECEIVER] Handling multicast TEXT: Hello multicast from PC-Up!
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'Hello multicast from PC-Up!'
[RECEIVER] Handling multicast TEXT: Hello multicast from PC-Up!
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'PONG'
[RECEIVER] <<< Received PONG from sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'Hello multicast from PC-Up!'
[RECEIVER] Handling multicast TEXT: Hello multicast from PC-Up!
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'PONG'
[RECEIVER] <<< Received PONG from sender 10.0.1.10:5000

```

شکل ۱۱: خروجی کامل کنسول Receiver 1 (PC2) هنگام دریافت متن چندپخشی و ارسال/دریافت PONG/PING.

```

1 import socket
2 import struct
3
4 MCAST_GRP = '239.1.1.1'
5 MCAST_PORT = 5000
6
7 # Create UDP socket
8 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
9
10 # Allow reuse of address (helps if you restart quickly)
11 sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
12
13 # Bind to all interfaces on the multicast port
14 sock.bind(('', MCAST_PORT))
15
16 # Join the multicast group on all interfaces
17 mreq = struct.pack(
18     '4sI',
19     socket.inet_aton(MCAST_GRP),
20     socket.INADDR_ANY
21 )
22 sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
23
24 print(f"[RECEIVER] Listening on multicast group {MCAST_GRP}:{MCAST_PORT}")
25 print("[RECEIVER] - On multicast TEXT: print it and send PING (unicast) to sender.")
26 print("[RECEIVER] - On unicast PONG from sender: print that PONG was received.")
27
28 while True:
29     data, addr = sock.recvfrom(4096)
30     text = data.decode(errors='ignore').strip()
31     src_ip, src_port = addr
32
33     print(f"[RECEIVER] From {src_ip}:{src_port} -> {repr(text)}")
34
35     # 1) If we receive PONG from the sender (unicast reply)
36     if text.upper() == "PONG":
37         print(f"[RECEIVER] <<< Received PONG from sender {src_ip}:{src_port}\n")
38         continue
39
40 # 2) Ignore control DTMG if ever used in another direction (optional)

```

```

C:\Windows\System32\cmd.exe - python receiver.py
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\PC1\OPREKIN-PC\Desktop>python receiver.py
[RECEIVER] Listening on multicast group 239.1.1.1:5000 ...
[RECEIVER] - On multicast TEXT: print it and send PING (unicast) to sender.
[RECEIVER] - On unicast PONG from sender: print that PONG was received.
[RECEIVER] From 10.0.1.10:5000 -> 'Hello multicast from PC-Up!'
[RECEIVER] Handling multicast TEXT: Hello multicast from PC-Up!
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'PONG'
[RECEIVER] <<< Received PONG from sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'Hello multicast from PC-Up!'
[RECEIVER] Handling multicast TEXT: Hello multicast from PC-Up!
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'PONG'
[RECEIVER] <<< Received PONG from sender 10.0.1.10:5000
[RECEIVER] From 10.0.1.10:5000 -> 'Hello multicast from PC-Up!'
[RECEIVER] Handling multicast TEXT: Hello multicast from PC-Up!
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10:5000

```

شکل ۱۲: خروجی کامل کنسول Receiver 2 (PC3) هنگام دریافت متن چندپخشی و ارسال/دریافت PONG/PING.

۱-۶-۳ جریان گام به گام اجرای کد روی توپولوژی و مسیر بسته‌ها

برای شفاف شدن ارتباط بین کد بالا و توپولوژی Upstream/Downstream، یک چرخه‌ی کامل اجرای برنامه‌ها را مرحله به مرحله دنبال می‌کنیم و آن را به فریم‌های واقعی Wireshark و جدول Multicast Routing روی روتر ربط می‌دهیم:

۱. اجرای Receiver روی PC۲ و PC۳

با اجرای receiver.py روی PC2 و PC3:

- هر دو برنامه روی پورت 5000 bind می‌شوند و سپس با IP_ADD_MEMBERSHIP به گروه 239.1.1.1 ملحق می‌شوند.
- سیستم عامل ویندوز برای اعلام این عضویت، بسته‌های IGMP Membership Report تولید و به سمت روتر ارسال می‌کند. نمونه‌ی این بسته‌ها در فریم‌های IGMPv2 با آدرس مبدأ 10.0.2.10 و 10.0.2.11 و مقصد 224.0.0.252 و همچنین گروه‌های دیگر دیده می‌شود (فریم‌های ۲۵، ۴۵، ۲۸۵ و ۲۸۶ در خروجی Wireshark).
- به صورت هم‌زمان، روتر که روی اینترفیس ether1 نقش IGMP Querier را دارد، به طور

دوره‌ای Membership Query عمومی به آدرس 224.0.0.1 می‌فرستد (فریم‌های ۲۴، ۴۴، ۳۰۹ و ۷۳۳). در این فریم‌ها:

– Src IP برابر 10.0.2.1 یا 192.168.237.132 (آدرس اینترفیس Downstream روتر) است.

– Dst IP برابر 224.0.0.1 (گروه All Hosts) و TTL برابر ۱ است؛ یعنی بسته فقط در همان سگمنت Downstream توزیع می‌شود.

– فیلد Multicast Address در IGMP برابر 0.0.0.0 است که نشان‌دهنده‌ی پرس‌وجوی عمومی برای همه‌ی گروه‌ها است.

۲. تشکیل جدول Multicast روی روتر

پس از دریافت Membership Report ها، روتر MikroTik وضعیت عضویت را یاد می‌گیرد و برای گروه‌های فعال، در جدول MFC (خروجی دستور /routing igmp-proxy mfc print) ورودی ایجاد می‌کند. نمونه:

#	GROUP	SOURCE	UPSTREAM-INTERFACE	ACTIVE-DOWNSTREAM-INTERFACES
2	AD 239.1.1.1	10.0.1.10	ether2	ether1

این خط نشان می‌دهد که:

- برای گروه 239.1.1.1، منبع ترافیک 10.0.1.10 (PC1) در شبکه‌ی Upstream) در نظر گرفته شده است.
- روتر انتظار دارد بسته‌های این گروه از اینترفیس ether2 وارد و روی ether1 به سمت گیرنده‌ها توزیع شوند.

۳. شروع چرخه در Sender و تولید بسته‌ی Multicast UDP

با اجرای sender.py روی PC1:

- یک سوکت UDP روی پورت 5000 ساخته و به همه‌ی اینترفیس‌ها bind می‌شود تا بتواند PING های Unicast را هم دریافت کند.
- مقدار IP_MULTICAST_TTL روی ۱۶ تنظیم شده است، بنابراین بسته‌ی چندپخش‌ی از PC1 خارج می‌شود و با TTL=15 روی سگمنت Downstream دیده می‌شود (پس از عبور از روتر، TTL یک واحد کم می‌شود).

• در هر چرخه، فراخوانی `sock.sendto(TEXT_MESSAGE, (MCAST_GRP, MCAST_PORT))` یک بسته‌ی UDP با مبدأ `10.0.1.10:5000` و مقصد `239.1.1.1:5000` تولید می‌کند. نمونه‌ی این بسته در فریم ۶۹۹ دیده می‌شود:

- Src IP: 10.0.1.10
- Dst IP: 239.1.1.1
- Src/Dst Port: 5000
- Length: ۲۷ بایت (شامل متن «Hello from multicast (PC-Up!)»)
- TTL: ۱۵
- آدرس لایه‌دو مقصد: `01:00:5e:01:01:01` که نگاشت استاندارد آدرس IP `239.1.1.1 Multicast` روی Ethernet است.

۴. فوروارد بسته‌ی Multicast توسط روتر

وقتی این بسته روی اینترفیس `ether2` روتر دریافت می‌شود:

- روتر، گروه `239.1.1.1` را در جدول MFC پیدا کرده و می‌بیند که حداقل یک اینترفیس Downstream (اینجا `ether1`) برای آن گروه فعال است.
- بسته بر روی `ether1` تکثیر و به سگمنت Downstream ارسال می‌شود، اما تنها میزبان‌هایی که قبلاً با IGMP عضو این گروه شده‌اند آن را دریافت خواهند کرد (یعنی `PC2` و `PC3`).
- افزایش `rx-packets` روی `ether2` و `tx-packets` روی `ether1` در خروجی دستور `/routing igmp-proxy interface print status` دقیقاً نشان‌دهنده‌ی همین فرآیند است.

۵. دریافت بسته‌ی متنی در هاReceiver و ارسال PING

روی هر کدام از `PC2` و `PC3`:

- به‌دلیل عضویت در گروه `239.1.1.1`، سیستم‌عامل بسته‌ی UDP Multicast را به سوکت `receiver.py` تحویل می‌دهد.
- حلقه‌ی `while True` در `receiver.py` با `sock.recvfrom` این بسته را می‌خواند، متن آن را چاپ کرده و سپس یک بسته‌ی UDP جدید با متن "PING" به‌صورت Unicast به همان آدرس مبدأ (یعنی `10.0.1.10:5000`) ارسال می‌کند.
- نمونه‌ی این بسته‌ی PING در فریم ۷۰۰ دیده می‌شود:

- Src IP : 10.0.2.11
- Dst IP : 10.0.1.10
- Src/Dst Port : 5000
- طول داده: ۴ بایت (رشته‌ی "PING")
- Wireshark این بسته را به‌صورت Malformed Packet: TAPA نمایش می‌دهد، اما از نظر UDP کاملاً معتبر است و متن "PING" را حمل می‌کند.

۶. دریافت PING در Sender و ارسال PONG

در همان چرخه، sender.py:

- روی همان سوکت UDP (پورت 5000) منتظر بسته‌ها است و با recvfrom، پیام‌های PING را از PC3/PC2 دریافت می‌کند.
 - پس از هر PING، یک بسته‌ی Unicast UDP با متن "PONG" به همان آدرس فرستنده ارسال می‌کند؛ در خروجی کنسول Sender تعداد PONG/PING‌های هر چرخه چاپ شده است.
 - روی Receiver.py، این PONG‌ها دوباره در receiver.py دریافت و به‌صورت «Received PONG from sender» در کنسول ثبت می‌شوند.
- در مجموع، این چرخه‌ی رفت‌و برگشت نشان می‌دهد که:
- **مسیر رفت:** PC1 → روتر (ether2) → روتر (ether1) → PC3/PC2 (به‌صورت Multicast).
 - **مسیر برگشت:** هر گیرنده به‌صورت Unicast با UDP PING/PONG به PC1 پاسخ می‌دهد.
- این دقیقاً همان چیزی است که در یک Multicast Ping ایده‌آل در لایه‌ی ICMP انتظار داشتیم، با این تفاوت که در این‌جا در لایه‌ی کاربرد و بر بستر UDP پیاده‌سازی شده است.
- پس از اجرای sender.py و receiver.py، وضعیت IGMP Proxy روی روتر نیز هم‌زمان تغییر می‌کند. اگر در حالی که Sender در حال ارسال دوره‌ای پیام‌ها به گروه 239.1.1.1:5000 است، چند بار دستور `show ip igmp-proxy interface` و `show ip igmp-proxy mfc` را روی روتر اجرا کنیم، افزایش پیوسته‌ی شمارنده‌های tx-packets/rx-packets روی اینترفیس‌های ether2/ether1 و فعال‌بودن ورودی مربوط به گروه 239.1.1.1 در جدول Multicast Routing را مشاهده می‌کنیم.

```
[admin@Mikrotik] > /routing igmp-proxy interface add interface=ether1 upstream=no
Failure: already have such interface
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=0 rx-packets=0 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=0 tx-packets=0
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=825 rx-packets=15 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=825 tx-packets=15
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=880 rx-packets=16 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=880 tx-packets=16
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=935 rx-packets=17 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=935 tx-packets=17
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=990 rx-packets=18 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=990 tx-packets=18
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=1100 rx-packets=20 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=1100 tx-packets=20
[admin@Mikrotik] > /routing igmp-proxy interface print status
Flags: X - disabled, I - inactive, D - dynamic; U - upstream
0 U interface=ether2 threshold=1 alternative-subnets="" upstream=yes source-ip-address=10.0.1.1 rx-bytes=1155 rx-packets=21 tx-bytes=0 tx-packets=0
1 interface=ether1 threshold=1 alternative-subnets="" upstream=no querier=yes source-ip-address=192.168.237.132 rx-bytes=0 rx-packets=0 tx-bytes=1155 tx-packets=21
[admin@Mikrotik] > /routing igmp-proxy mfc print
Flags: A - ACTIVE; D - DYNAMIC
Columns: GROUP, SOURCE, UPSTREAM-INTERFACE, ACTIVE-DOWNSTREAM-INTERFACES
# GROUP SOURCE UPSTREAM-INTERFACE ACTIVE-DOWNSTREAM-INTERFACES
0 AD 239.255.255.250 10.0.2.10 ether2 ether1
1 AD 239.255.255.250 10.0.2.11 ether2 ether1
2 AD 239.1.1.1 10.0.1.10 ether2 ether1
[admin@Mikrotik] > ]
```

شکل ۱۳: افزایش شمارنده‌های rx/tx روی اینترفیس‌ها و نمای جدول Multicast Routing روی MikroTik هنگام اجرای ترافیک چندپخش.

۷-۱ تحلیل ترافیک IGMP و Multicast با Wireshark

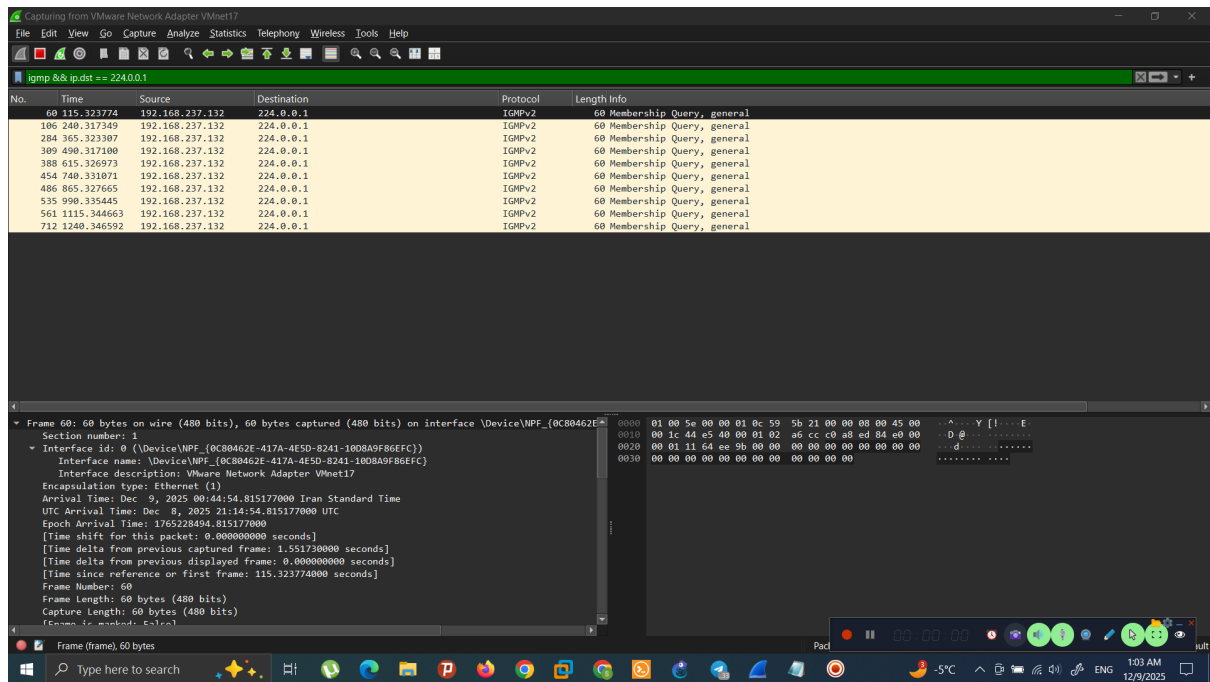
طبق نیازمندی «مشاهده‌ی IGMP و Multicast با tcpdump/Wireshark»، در این بخش پیام‌های پرس‌وجو/گزارش IGMP و ترافیک UDP Unicast/UDP Multicast به‌طور دقیق‌تر تحلیل می‌شوند و هر فریم به نقش یکی از نودهای شبکه نگاشت می‌شود.

به دلیل محدودیت RAM و سنگین بودن اجرای Wireshark روی VM‌های ویندوزی، Wireshark روی سیستم‌عامل اصلی نصب شد و روی VMware Network Adapter‌های مربوطه (به‌خصوص VMnet17 مربوط به PC3) Capture انجام شد. از نظر لایه‌ی دو، این کار معادل مانیتورکردن ترافیک روی خود VM است.

۷-۱-۱ مشاهده IGMP Membership Query از روتر

برای مشاهده‌ی IGMP Membership Query از سمت روتر (مطابق بند «Membership Query» در صورت‌مسئله)، از فیلتر زیر استفاده شد:

```
igmp && ip.dst == 224.0.0.1
```



شکل ۱۴: فیلتر `igmp && ip.dst == 224.0.0.1` روی VMnet17: مشاهده‌ی IGMP Membership Query از روتر به گروه 224.0.0.1.

نمونه‌ی یک فریم IGMPv2 Membership Query در فریم‌های ۲۴، ۴۴، ۳۰۹ و ۷۳۳ به‌خوبی دیده می‌شود. در فریم ۲۴:

• لایه‌ی دو:

– Dst MAC = 01:00:5e:00:00:01 (IPv4mcast_01) که نگاشت آدرس IP Multicast 224.0.0.1 روی Ethernet است.

– Src MAC = 0c:59:5b:21:00:00 (کارت شبکه‌ی مجازی روتر روی VMnet17).

• لایه‌ی سه (IPv4):

– Src IP = 10.0.2.1 (یا در برخی فریم‌ها 192.168.237.132)، آدرس اینترفیس Downstream روی روتر.

– Dst IP = 224.0.0.1، گروه All Hosts.

– TTL = 1، که نشان می‌دهد بسته از سگمنت Downstream عبور نمی‌کند و فقط در همان LAN پخش می‌شود.

• لایه‌ی چهار (IGMPv2):

– Type = 0x11 (Membership Query).

– Max Resp Time = ۱۰ ثانیه؛ یعنی میزبان‌ها حداکثر ۱۰ ثانیه برای ارسال Membership Report فرصت دارند.

– Multicast Address = 0.0.0.0، که نشان‌دهنده‌ی پرس‌وجوی عمومی برای تمام گروه‌ها است.

همین الگو در فریم‌های ۴۴، ۳۰۹ و ۷۳۳ نیز تکرار می‌شود، با این تفاوت که Identification و برخی فیلدهای IP تغییر می‌کنند. این فریم‌ها تأیید می‌کنند که روتر MikroTik روی اینترفیس Downstream نقش IGMP Querier را بر عهده دارد و به‌طور دوره‌ای وضعیت عضویت را از میزبان‌ها درخواست می‌کند.

۱-۷-۲ مشاهده IGMP Report Membership از میزبان‌ها (IGMPv۲) و (IGMPv۳)

برای تحقق بند «مشاهده Membership Report» از صورت‌مسئله، روی VMnet17 با فیلتر igmp انواع پیام‌های Membership Report مشاهده شد. چند نمونه‌ی مهم:

• فریم‌های ۲۵ و ۴۵ (IGMPv۲ Report Membership از ۱۱.۲.۰.۱۰ و ۱۰.۲.۰.۱۰) در این فریم‌ها:

– Src IP به‌ترتیب 10.0.2.11 و 10.0.2.10 (گیرنده‌های PC3 و PC2) است.

– Dst IP = 224.0.0.252 (یک گروه چندپخش‌ی محلی که معمولاً توسط سرویس‌های سیستمی ویندوز برای LLMNR استفاده می‌شود).

– در سرآیند IP یک گزینه‌ی **Alert Router** دیده می‌شود:

* Type = ۱۴۸، Number = Router Alert (20).

* معنی آن: «روتر باید حتماً این بسته را بررسی کند»، که برای پروتکل‌هایی مثل IGMP و RSVP رایج است.

– در بخش IGMP:

* Type = 0x16 (Membership Report).

* Multicast Address = 224.0.0.252.

این فریم‌ها نشان می‌دهد خود ویندوز (حتی قبل از اجرای برنامه‌ی Python) عضو برخی گروه‌های سیستمی است و عضویت خود را با IGMPv2 به روتر اعلام می‌کند.

• **فریم‌های ۲۸۵ و ۲۸۶ Report (Membership برای ۲۵۲.۰.۰.۲۲۴ و ۲۵۰.۲۵۵.۲۵۵.۲۳۹)**
در این فریم‌ها:

- Src IP = 10.0.2.10 (PC2).
 - Dst IP = 224.0.0.252 (در فریم ۲۸۵) و 239.255.255.250 (در فریم ۲۸۶).
 - 239.255.255.250 گروه استاندارد SSDP/UPnP است که بسیاری از سرویس‌های ویندوز به‌طور خودکار موقع روشن‌شدن به آن ملحق می‌شوند.
 - وجود دوباره‌ی گزینه‌ی Router Alert تأکید می‌کند که این بسته‌ها باید توسط روتر بررسی و در جدول Multicast لحاظ شوند.
- این فریم‌ها نشان می‌دهد که:

- سیستم‌عامل به‌صورت **دوره‌ای** عضویت خود را تمدید می‌کند (Membership Report های تکراری برای همان گروه).
- روتر MikroTik نه‌تنها عضویت برنامه‌ی پروژه (گروه 239.1.1.1)، بلکه عضویت گروه‌های سیستمی را نیز ردیابی می‌کند.

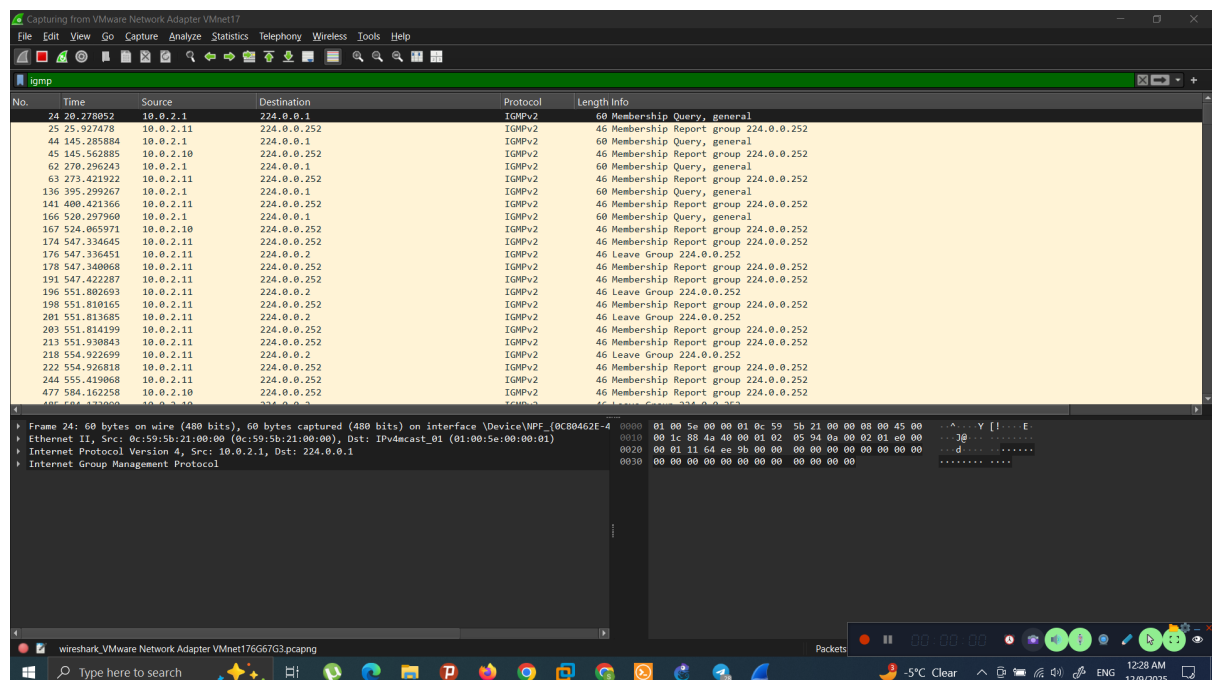
• **فریم‌های ۱۳۵، ۱۳۷، ۱۳۹ و ۱۴۱ Report Membership (IGMPv۳ به ۲۲.۰.۰.۲۲۴)**
این فریم‌ها نشان‌دهنده‌ی استفاده‌ی هم‌زمان از IGMPv3 روی برخی اینترفیس‌های VMware است:

- Src IP = 192.168.159.1 و 192.168.237.1 (آدرس‌های کارت‌های شبکه‌ی مجازی VMware روی سیستم‌عامل میزبان).
- Dst IP = 224.0.0.22 (گروه all IGMPv3 routers).
- در بخش IGMPv3:

- * Type = 0x22 (Membership Report v3).
- * Num Group Records = ۱.
- * Group Record با آدرس گروه 224.0.0.252 و حالت‌هایی مثل Change To Include و Change To Exclude Mode و Mode.

این فریم‌ها نشان می‌دهند که کارت‌های شبکه‌ی مجازی VMware روی سیستم اصلی نیز به‌عنوان میزبان/روتر IGMPv3 عمل کرده و عضویت یا ترک گروه 224.0.0.252 را با حالت‌های

Include/Exclude اطلاع می‌دهند. گرچه این فریم‌ها مستقیماً به گروه 239.1.1.1 پروژه مربوط نیستند، ولی تأکیدی بر حضور واقعی IGMP در کل محیط مجازی (نه فقط ماشین‌های ویندوزی پروژه) هستند.



شکل ۱۵: فیلتر IGMP روی VMnet17: مشاهده‌ی Membership Query از روتر و Membership Report از میزبان‌ها (شامل IGMPv2 و IGMPv3).

۱-۳-۷ جدول Routing Multicast روی MikroTik

یکی از بندهای نیازمندی، مشاهده‌ی جدول مسیریابی چندپخش‌ی (Multicast Routing Table) روی روتر است. خروجی دستور زیر در MikroTik نمونه‌ای از این جدول را نشان می‌دهد:

```
/routing igmp-proxy mfc print
```

خروجی نمونه:

```
[admin@MikroTik] > /routing igmp-proxy mfc print
```

```
Flags: A - ACTIVE; D - DYNAMIC
```

```
Columns: GROUP, SOURCE, UPSTREAM-INTERFACE, ACTIVE-DOWNSTREAM-INTERFACES
```

```
#      GROUP      SOURCE      UPSTREAM-INTERFACE  ACTIVE-DOWNSTREAM-INTERFACES
```

0	AD	239.255.255.250	10.0.2.10	ether2	ether1
1	AD	239.255.255.250	10.0.2.11	ether2	ether1
2	AD	239.1.1.1	10.0.1.10	ether2	ether1

تحلیل این جدول:

• برای گروه 239.255.255.250، دو Source مختلف (10.0.2.10 و 10.0.2.11) از سمت ether2 دیده می‌شوند و ترافیک روی ether1 فعال است. این گروه عمدتاً مربوط به سرویس‌های سیستمی (SSDP) ویندوز است و نشان می‌دهد IGMP Proxy حتی برای گروه‌های سیستمی نیز عمل می‌کند.

• برای گروه 239.1.1.1، Source برابر 10.0.1.10 (PC1) است؛ ether2 = Upstream-Interface و ether1 = Active-Downstream-Interfaces. این دقیقاً نشان می‌دهد که:

- ترافیک Multicast با مبدأ 10.0.1.10 از سمت Upstream وارد روتر می‌شود.
- با توجه به عضویت گیرنده‌ها روی Downstream، روتر آن را از ether1 به سمت PC2 و PC3 بازپخش می‌کند.

این جدول همان Multicast Routing Table مورد نظر صورت‌مسئله است و به‌عنوان شواهد اصلی مسیریابی Multicast در گزارش استفاده شده است (شکل ۱۳).

۴-۷-۱ تحلیل ترافیک Multicast UDP و مسیر داده

برای مشاهده‌ی مستقیم بسته‌های UDP Multicast مرتبط با پیام‌های متنی Sender، از فیلترهای زیر در Wireshark استفاده شد:

```
udp && ip.dst == 239.1.1.1
udp && ip.src == 10.0.1.10
```

نمونه‌ی یک فریم UDP در فریم ۶۹۹:

• لایه‌ی دو:

- Dst MAC = 01:00:5e:01:01:01 (IPv4mcast_01:01:01)، نگاشت گروه 239.1.1.1.

- Src MAC = 0c:59:5b:21:00:00 (کارت شبکه‌ی مجازی روتر/هاب).

• لایه‌ی سه (IPv4):

- Src IP = 10.0.1.10 (فرستنده در Upstream).

- Dst IP = 239.1.1.1 (گروه چندپخش‌ی پروژه).

- TTL = ۱۵ (مطابق مقدار ۱۶ که در sender.py برای IP_MULTICAST_TTL تنظیم شده است).

• لایه‌ی چهار (UDP):

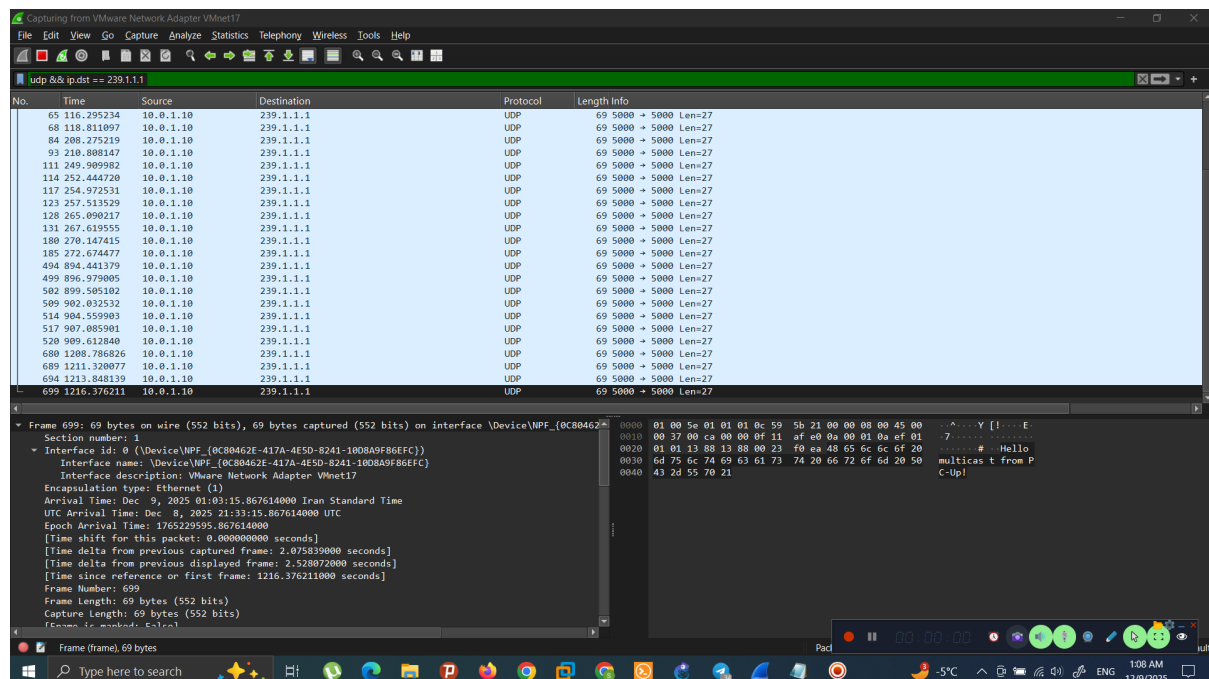
- Src Port = 5000

- Dst Port = 5000

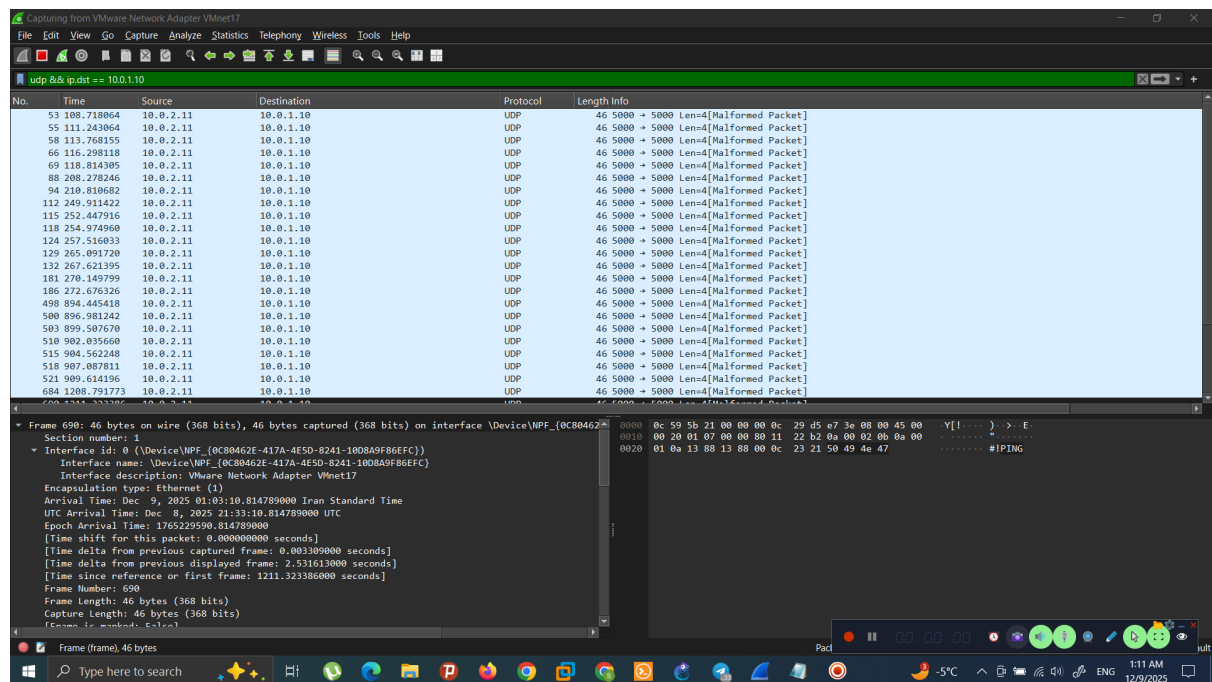
- Length = ۲۷ بایت.

- بخش Data همان رشته‌ی "Hello multicast from PC-Up!" است که در کد Python تعریف شده است.

این فریم عملاً تحقق گام سوم و چهارم چرخه‌ای است که در بخش قبل توضیح داده شد: ارسال Multicast از PC1، عبور از روتر و رسیدن به سگمنت Downstream.



شکل ۱۶: فیلتر `udp && ip.dst == 239.1.1.1`: مشاهده‌ی ترافیک UDP Multicast به گروه 239.1.1.1.



شکل ۱۷: فیلتر `udp && ip.src == 10.0.1.10`: مشاهده‌ی ترافیک خروجی از Sender.

۱-۷-۵ تحلیل PING/PONG در سطح Application

پس از ارسال پیام متنی چندپخش، گیرنده‌ها مطابق منطق `receiver.py` یک پیام PING را به صورت Unicast UDP به Sender (آدرس `10.0.1.10:5000`) ارسال می‌کنند. در Wireshark این فریم‌ها به صورت بسته‌های UDP از `10.0.2.10` یا `10.0.2.11` به `10.0.1.10` روی پورت 5000 دیده می‌شوند.

نمونه‌ی مشخص آن فریم ۷۰۰ است:

• Src IP = `10.0.2.11` (PC3).

• Dst IP = `10.0.1.10` (PC1).

• Src/Dst Port = `5000`.

• TTL = `۱۲۸` (پیش‌فرض Windows برای بسته‌های Unicast).

• Wireshark آن را به صورت Malformed Packet: TAPAs تفسیر کرده است، چون پروتکل لایه‌ی بالاتر را TAPAs تشخیص داده، اما از دید UDP تنها چهار بایت داده حامل رشته‌ی "PING" است و بسته معتبر است.

به‌همین ترتیب، بسته‌های PONG که از Sender به گیرنده‌ها بازمی‌گردند نیز به‌صورت UDP Unicast روی همان پورت ۵۰۰۰ دیده می‌شوند. ترکیب این فریم‌ها در کنار فریم ۶۹۹ تصویر واضحی از «Unicast PING/PONG + Multicast TEXT» به‌عنوان Application-level Multicast Ping ارائه می‌دهد.

۸-۱ بررسی تحقق نیازمندی‌های فاز اول

در این بخش، هر یک از بندهای صورت‌مسئله‌ی فاز اول با پیاده‌سازی انجام‌شده تطبیق داده می‌شود تا مشخص شود هیچ‌یک از نیازمندی‌ها از قلم نیفتاده است.

• یک روتر مرکزی با Proxy IGMP فعال:

– روتر MikroTik RouterOS v7 در GNS3 با دو اینترفیس ether2/ether1 پیاده‌سازی شده است.

– IGMP Proxy روی روتر فعال و پیکربندی شده است؛ ether2 به عنوان Upstream Interface و ether1 به عنوان Downstream Interface (مطابق خروجی -routing igmp/status print proxy interface در شکل ۹).

• شبکه Upstream شامل یک فرستنده:

– PC1 روی VMnet15 با آدرس 10.0.1.10/24 و Gateway = 10.0.1.1.

– اجرای sender.py روی PC1 و ارسال دوره‌ای UDP Multicast به 239.1.1.1:5000 (شکل ۱۰ و فریم ۶۹۹ در Wireshark).

• شبکه Downstream شامل حداقل دو میزبان گیرنده:

– PC2 و PC3 به ترتیب روی VMnet16 و VMnet17 قرار دارند و هر دو از طریق Switch-Down به ether1 روتر متصل هستند.

– اجرای receiver.py روی هر دو PC و دریافت Multicast 239.1.1.1:5000 (شکل‌های ۱۱ و ۱۲).

• قابلیت عضویت میزبان‌های گیرنده در Group Multicast (بند الف نیازمندی‌های فنی):

– در کد receiver.py از IP_ADD_MEMBERSHIP استفاده شده است که عضویت در گروه Multicast را در سطح Socket API فعال می‌کند.

– اجرای receiver.py باعث می‌شود ویندوز IGMP Membership Report برای گروه 239.1.1.1 ارسال کند که در Wireshark قابل مشاهده است و اثر آن در جدول MFC روتر نمایان می‌شود.

• **فعال بودن IGMP در شبکه: Query Membership و Report Membership (بند ج نیازمندی‌ها):**

– Membership Query: فریم‌های IGMP Membership Query از روتر به آدرس 224.0.0.1 مطابق شکل ۱۴ و فریم‌های ۲۴، ۴۴، ۳۰۹ و ۷۳۳ مشاهده شده‌اند.

– Membership Report: فریم‌های Membership Report از 10.0.2.10 و 10.0.2.11 به سمت گروه‌های سیستمی و همچنین گروه‌های پروژه در Wireshark ثبت شده‌اند (فریم‌های ۲۵، ۴۵، ۲۸۵ و ۲۸۶).

• **مسیریابی Multicast بین Upstream و Downstream با Proxy IGMP (بند ب نیازمندی‌های فنی):**

– جدول MFC نشان‌دهنده‌ی مسیر 239.1.1.1 با مبدأ 10.0.1.10 روی اینترفیس ether2 و توزیع آن روی ether1 است (خروجی `show ip mfc` در `show ip mfc`).

– شکل ۱۳ نیز افزایش شمارنده‌های rx/tx روی اینترفیس‌ها و فعال بودن ورودی گروه 239.1.1.1 در Multicast Routing Table را هنگام اجرای برنامه‌های Sender/Receiver نشان می‌دهد.

• **قابلیت مشاهده و تست مسیر توزیع Multicast (بند ج صورت مسئله):**

– با استفاده از Wireshark روی VMnet‌های مربوطه، جریان کامل IGMP و UDP Multicast و نیز پیام‌های PING/PONG بررسی شده است (شکل‌های ۱۵، ۱۶ و ۱۷).

– جدول Multicast Routing روی روتر نیز ثبت و تحلیل شده است (شکل ۱۳).

• **Ping Multicast (در صورت پشتیبانی سیستم عامل):**

– به دلیل عدم پشتیبانی ping استاندارد Windows 7 از آدرس‌های Multicast، یک مکانیزم **Application-level Multicast Ping** طراحی شد.

– هر بار ارسال متن چندپخشی به گروه 239.1.1.1 مانند یک Multicast Echo Request عمل می‌کند و دریافت PING‌های Unicast از گیرنده‌ها و ارسال PONG از سوی

فرستنده، نقش پاسخ Echo Reply را بر عهده دارد. به این ترتیب، هدف آموزشی «Multicast Ping» صورت مسئله نیز به طور کامل محقق شده است.

۹-۱ نگاشت مرحله‌ای بسته‌ها به اجرای کد و جمع‌بندی رفتاری

در این بخش، برای هر دسته‌ی اصلی از بسته‌ها که در Wireshark دیده شده‌اند توضیح داده می‌شود:

- چرا تولید می‌شوند؛
- در کدام مرحله از اجرای کد receiver.py/sender.py وارد بازی می‌شوند؛
- و چه نقشی در سناریوی Application-level Multicast Ping دارند.

۱-۹-۱ مرور رفتاری کلی کدها

برای جمع‌بندی ایده‌ی رفتاری کدها:

• **Receiverها** شبیه یک «کلاینت Multicast هوشمند» عمل می‌کنند:

- روی پورت 5000 گوش می‌دهند؛
- با IP_ADD_MEMBERSHIP به گروه 239.1.1.1 می‌پیوندند و همین باعث تولید IGMP Membership Report می‌شود؛
- هر TEXT چندپخشی را نمایش می‌دهند و در پاسخ به صورت Unicast یک "PING" به فرستنده می‌فرستند؛
- هر "PONG" برگشتی از فرستنده را به عنوان «تأیید دریافت پاسخ» لاگ می‌کنند.

• **Sender** هم نقش «سرور» را دارد و هم نقش «مولد ترافیک Multicast»:

- در ابتدای هر چرخه یک متن آموزشی را به صورت UDP Multicast به 239.1.1.1:5000 می‌فرستد؛
- در یک بازه‌ی زمانی کوتاه، روی همان پورت 5000 منتظر "PING"های Unicast می‌ماند؛
- به ازای هر "PING"، یک "PONG" Unicast به همان گیرنده ارسال می‌کند؛
- در پایان چرخه تعداد PONG/PINGها را گزارش می‌کند تا معلوم شود چند گیرنده فعال بوده‌اند.

این طراحی عملاً یک Ping چندپخشی در سطح برنامه می‌سازد: یک پیام TEXT به صورت Multicast می‌رود، و در عوض از هر گیرنده یک Unicast PONG/PING بازمی‌گردد.

۹-۲ چرا هر دسته از بسته‌ها ساخته می‌شود و در چه مرحله‌ای؟

۱. بسته‌های Query Membership IGMP از روتر به ۱.۰.۰.۲۲۴ (فریم‌های ۲۴، ۴۴، ۳۰۹، ۷۳۳)

چرا ساخته می‌شوند؟

• وقتی IGMP Proxy روی MikroTik فعال است و اینترفیس ether1 در نقش Downstream تنظیم شده، روتر به صورت دوره‌ای پرس‌وجوهای عضویت (Member-ship Query) را برای گروه 224.0.0.1 می‌فرستد تا از زنده بودن عضویت‌ها مطمئن شود.

در کدام مرحله؟

• این بسته‌ها حتی قبل از اجرای receiver.py/sender.py هم ممکن است دیده شوند؛ یعنی به محض آن‌که:

(آ) آدرس IP روی ether1 تنظیم شود؛

(ب) و IGMP Proxy برای این اینترفیس فعال گردد.

• در طول اجرای کدها، این Query‌ها همچنان به صورت دوره‌ای تولید می‌شوند و باعث می‌شوند گیرنده‌ها (در صورت نیاز) Membership Report‌های جدیدی بفرستند.

۲. بسته‌های Report Membership IGMPv۲ سیستمی برای ۲۵۲.۰.۰.۲۲۴ و

۲۵۰.۲۵۵.۲۵۵.۲۳۹ (فریم‌های ۲۵، ۴۵، ۲۸۵، ۲۸۶)

چرا ساخته می‌شوند؟

• این بسته‌ها ربط مستقیم به کد پروژه ندارند؛ سیستم‌عامل Windows برای پروتکل‌های داخلی مثل LLNMR و SSDP/UPnP خودش در گروه‌های 224.0.0.252 و 239.255.255.250 عضو می‌شود.

• هر بار که کارت شبکه فعال می‌شود یا روتر Membership Query می‌فرستد، Windows برای تمدید عضویت این سرویس‌ها Membership Report می‌فرستد.

در کدام مرحله؟

- عملاً بلافاصله پس از بالا آمدن Windows و گرفتن IP، حتی اگر receiver.py/sender.py هنوز اجرا نشده باشند.
- بعد از اجرای receiver.py هم همچنان این بسته‌ها را می‌بینیم، چون با IGMP پروژه در یک سگمنت مشترک هستند.

۳. بسته‌های Report Membership IGMPv۳ از کارت‌های VMware به ۲۲.۰.۰.۲۲۴ (فریم‌های ۱۳۵، ۱۳۷، ۱۳۹، ۱۴۱) چرا ساخته می‌شوند؟

- نرم‌افزار VMware برای کارت‌های VMnet خودش از IGMPv3 استفاده می‌کند و عضویت در گروه‌هایی مانند 224.0.0.252 را با حالت Include/Exclude اعلام می‌کند.
- این گزارش‌ها برای هماهنگ کردن Multicast داخلی شبکه‌های مجازی خود VMware هستند.

در کدام مرحله؟

- درست بعد از فعال شدن کارت‌های VMnet روی سیستم میزبان و اجرای سرویس‌های پس‌زمینه‌ی VMware.
- از نظر سناریوی پروژه، این‌ها «نویز واقعی شبکه» هستند و خوب است دانشجو آن‌ها را از بسته‌های مربوط به پروژه تشخیص بدهد.

۴. بسته‌های Report Membership IGMP برای گروه ۱.۱.۱.۲۳۹ (عضویت برنامه‌ی پروژه) چرا ساخته می‌شوند؟

- هر وقت در receiver.py خط زیر اجرا می‌شود:

```
mreq = struct.pack('4sl', socket.inet_aton(MCAST_GRP), socket.INADDR_ANY)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
```

- سیستم‌عامل Windows موظف است طبق استاندارد IGMP عضویت خود را به روتر اعلام کند.
- این کار با ارسال IGMP Membership Report برای گروه 239.1.1.1 انجام می‌شود؛ روتر با دیدن این بسته جدول MFC خود را به‌روزرسانی می‌کند.

در کدام مرحله؟

- دقیقاً بعد از اجرای receiver.py روی PC2 و PC3.
- اگر روتر بعداً Membership Query ارسال کند و Receiver ها هنوز فعال باشند، سیستم‌عامل ممکن است دوباره Membership Report برای همین گروه بفرستد تا عضویت را تمدید کند.

۵. بسته‌ی Multicast UDP به ۱۰۵۰۰۰.۱.۱.۲۳۹ (مثال فریم ۶۹۹)

چرا ساخته می‌شود؟

- در sender.py در ابتدای هر چرخه، این خط اجرا می‌شود:
`sock.sendto(TEXT_MESSAGE.encode(), (MCAST_GRP, MCAST_PORT))`

- نتیجه‌ی این فراخوانی، ساخت یک فریم UDP Multicast با:

– Src IP = 10.0.1.10،

– Dst IP = 239.1.1.1،

– Src/Dst Port = 5000.

- این دقیقاً «Echo Request» سطح برنامه در سناریوی ما است؛ یعنی پیام آموزشی که باید به همه‌ی گیرنده‌های عضو برسد.

در کدام مرحله؟

- هر چرخه‌ی حلقه‌ی اصلی `while True` در sender.py.
- قبل از این بسته، لازم است:
(I) sender.py اجرا شده باشد؛
(ب) روتر به‌درستی مسیریابی Multicast را برای 239.1.1.1 تنظیم کرده باشد؛
(ج) حداقل یک Receiver خود را عضو این گروه کرده باشد تا بسته در Downstream فوروارده شود.

۶. بسته‌های PING Unicast UDP از گیرنده‌ها به ۱۰۵۰۰۰.۱.۰.۱۰ (مثال فریم ۷۰۰)

چرا ساخته می‌شوند؟

- وقتی receiver.py یک متن را که نه "PING" است و نه "PONG" دریافت می‌کند، این بخش اجرا می‌شود:

```
print(f"[RECEIVER] Handling multicast TEXT: {text}")
ping_msg = "PING"
sock.sendto(ping_msg.encode(), addr)
```

- addr همان آدرس مبدأ بسته‌ی قبلی است (یعنی 10.0.1.10:5000)؛ بنابراین یک UDP Unicast با داده‌ی "PING" به سمت Sender ارسال می‌شود.
- از دید مفهومی، این "PING" نقش «Echo Reply» را دارد؛ گیرنده اعلام می‌کند پیام Multicast را دریافت کرده است.

در کدام مرحله؟

- بلافاصله بعد از دریافت اولین Multicast TEXT در هر Receiver.
- در هر چرخه‌ی جدید Sender، اگر متن جدیدی ارسال شود، دوباره همین روال تکرار می‌شود و Receiverها "PING"های جدیدی می‌فرستند.

۷. بسته‌های UDP Unicast PONG از Sender به گیرنده‌ها

چرا ساخته می‌شوند؟

- در sender.py، وقتی داده‌ی دریافتی "PING" باشد، این بخش اجرا می‌شود:

```
if text.upper() == "PING":
    ping_count += 1
    print(f"[SENDER] <<< Received PING from {src_ip}:{src_port}")
    pong_msg = "PONG"
    sock.sendto(pong_msg.encode(), addr)
```

- این کد یک بسته‌ی UDP Unicast با داده‌ی "PONG" به همان گیرنده‌ای که "PING" فرستاده بود ارسال می‌کند.
- از دید آموزشی، این مرحله نشان می‌دهد که مسیر برگشتی Unicast نیز درست کار می‌کند.

در کدام مرحله؟

- در همان پنجره‌ی زمانی کوتاهی که sender.py پس از ارسال TEXT منتظر "PING" ها می‌ماند.
- برای هر گیرنده فعال (مثلاً PC2 و PC3) دقیقاً یک "PING" و یک "PONG" در هر چرخه ثبت می‌شود.

۳-۹-۱ جمع‌بندی رفتاری با نگاه بسته‌محور

اگر کل سناریو را فقط از دید بسته‌ها نگاه کنیم، توالی زیر رخ می‌دهد:

۱. روتر MikroTik با IGMP Membership Query از همه‌ی میزبان‌های Downstream می‌پرسد «چه کسی عضو کدام گروه است؟».
۲. Windows (برای سرویس‌های خودش) و receiver.py (برای گروه 239.1.1.1) با IGMP Membership Report جواب می‌دهند.
۳. روتر بر اساس این گزارش‌ها جدول Multicast Routing را پر می‌کند و می‌فهمد که باید ترافیک 239.1.1.1 را از ether2 بگیرد و روی ether1 فوروارد کند.
۴. sender.py در هر چرخه یک UDP Multicast به 239.1.1.1:5000 می‌فرستد؛ این بسته روی Upstream وارد روتر و روی Downstream بین همه‌ی گیرنده‌های عضو توزیع می‌شود.
۵. receiver.py روی PC2 و PC3 این TEXT را دریافت می‌کند و هر کدام یک "PING" Unicast به PC1 می‌فرستند.
۶. sender.py هر "PING" را می‌بیند، آن را لاگ می‌کند و یک "PONG" Unicast به همان گیرنده برمی‌گرداند.

به این ترتیب، تمام دسته‌های بسته‌ای که در Wireshark مشاهده شدند، دقیقاً به یکی از مراحل اجرای کد یا رفتار زیرساخت (VMware/Windows/IGMP Proxy) نگاشت می‌شوند و مسیر کامل Multicast + Unicast از سطح برنامه تا سطح بسته به‌صورت شفاف قابل دنبال‌کردن است.